INTERNAL PREP MATERIAL - NOT A SLIDE

Q Grid - Judge Q&A; Prep Sheet

Hackathon pitch defense - Quantum Optimization for EV Charging Stations - keep these numbers at your fingertips.

The 30-second answer. Q Grid is pure software that sits between every EV and every charger and decides who charges where and when. A tiny agent in each car reports GPS and battery level; our Center turns the whole city into one optimization problem - a 344-binary QUBO - and solves it with quantum plus classical solvers. In a reproducible Bengaluru simulation (seed 42), it cuts average waiting 21% versus today's first-come-first-served, with zero capacity violations, while 74.4% of the energy is billed in the cheap solar midday band. Drivers never open an app - charging is a background task. The people who fund and build Q Grid run the console.

Numbers cheat sheet (all from one reproducible run: python3 engine/qgrid_engine.py, seed 42)

Metric	Value	Where it comes from
Fleet / network	15 agents, 5 stations, 6 ports, 12 half-hour slots (12:00-17:30)	engine scenario, Bengaluru zones
QUBO size	344 binaries (session-start y[v,s,tau])	build_qubo(); slack-encoded capacity
Greedy FCFS total	Rs 4,206 - avg wait 0.93 slots	baseline solve, 0.1 ms
Simulated Annealing	Rs 4,180 - avg wait 0.73 slots (-21% wait cost)	400k iters, 2.0 s laptop CPU, production path
QAOA (crunch cell)	Rs 628 vs exact optimum Rs 619 (ratio 1.015)	20 qubits, p=1, COBYLA, 141 s
In-cell greedy	Rs 698 - QAOA beats it by 10%	fair cell-level baseline
Renewable share	74.4% of 352.3 kWh in solar-rich midday band	tariff Rs 4.00-4.50 vs Rs 18.00 evening
Violations	0 - on every shipped plan	feasibility guard + repair pass

Every number above is printed in results/results.json and results/run.log in the repo - hand them to judges if they want to verify.

Technical questions

Q1. Walk me through the QUBO. Why these variables?

One binary y[v,s,tau] per (vehicle, candidate station, start slot) = 1 exactly when vehicle v begins a single continuous charging session at station s in half-hour slot tau. The objective is a straight rupee sum: energy E(v,s) times tariff(tau), plus travel at Rs 9/km, plus a waiting cost w(v) per delayed slot where our AI triage sets w to Rs 260, 150 or 80 for NOW, SOON and CAN-WAIT cars. Two constraint families complete it: each vehicle charges exactly once (quadratic penalty, lambda = 4000) and port capacity is enforced as an exact inequality using slack variables weighted 1..ports. We deliberately moved away from flat per-slot variables because they silently distorted capacity by the max port count.

Q2. Why QAOA at all - isn't simulated annealing enough?

SA is our production path and it ships the plan in 2 seconds; we never hide that. QAOA is dispatched to the crunch cell - the 20-qubit sub-problem where stations actually collide - because that is where scheduling quality is won or lost as a city scales. The assignment problem is combinatorial, and heuristic quality degrades unpredictably exactly in those contested cells, while QAOA maps natively onto the QUBO/Ising form real hardware runs. Today we already measure the gap: QAOA lands within 1.5% of the brute-force optimum and beats the in-cell greedy baseline by 10%.

Q3. What does "approximation ratio 1.015" mean?

We brute-force enumerate all 2^{20} assignments of the crunch cell, which gives the provably optimal cost Rs 619. QAOA's best feasible sample evaluates to Rs 628, and 628/619 = 1.015 - so 1.000 would be perfect and we are within 1.5% of perfect. Only 17% of raw quantum samples were feasible; a repair pass and a feasibility guard guarantee the shipped plan is always valid. Quantum proposes, classical verifies.

Q4. Fifteen cars and Rs 26 saved - why should we care?

Because the mechanism, not the absolute rupees, is what we demoed: urgency-weighted waiting, exact capacity encoding, and a solver stack that provably beats the FCFS default on the same cost function. Congestion in our scenario is mild by design - we tuned it to be honest, not heroic. The money gets real when contested cells have 100+ vehicles: fleet depots, highway plazas, V2G arbitrage against demand-response signals. Results.json and run.log are the audit trail for everything we claim.

Q5. How does the capacity encoding work?

For each station and slot we need "occupied ports is less than or equal to ports". A naive penalty punishes empty ports too, which once made SA overfill stations and lose to greedy - our own debugging story. The fix: exact slack encoding, where we add binaries u weighted m = 1..ports so that occupied + u = ports exactly. Every feasible occupancy pattern has exactly one slack assignment, so the optimizer is never rewarded for filling ports artificially.

Q6. What is the solve-time budget in production?

The classical path never blocks: greedy is 0.1 ms and SA is 2.0 s on a laptop CPU for the full grid, recomputed every 30-minute window. QAOA on the crunch cell took 141 s in our run - that runs out-of-band as an upgrade pass, and if it is late or down, the SA plan ships anyway. Layered response is deliberate: quantum improves the answer, it is never a dependency.

Product, community and money

Q7. Who is the customer, and who pays?

Three seats at the table. Operators and fleet owners pay for coordination at scale - depots and charging networks where 21% less waiting translates directly into throughput. OEMs integrate the free, open Q-Agent SDK because it makes their vehicles smarter without hardware. And the community: contributors and shareholders fund regional Centers the way early cloud infrastructure was funded by operators, not car owners - and they get the operator console and a governance voice in return.

Q8. Explain the community-funded model in one breath.

The engine, the QUBO math and this console are open source under Apache-2.0 plus our Reciprocity Addendum: anyone may use it, rebuild it, even sell it - but whoever sells owes the project a private copy of their improved version. People who believe in it buy in; that capital builds regional Centers; contributors and shareholders run the same console you just saw, with founder aghosh (JustCode) and Qub3RAM.team leading. Drivers pay nothing and see nothing - Q Grid is background infrastructure for them. Formal share issuance gets structured with counsel at incorporation - today this is a governance and funding blueprint, stated honestly.

Q9. Why do drivers need no app at all?

Because the agent is pre-installed and the decisions are made for them: the SDK reports position and battery, the Center answers with "plug in here, at 13:00, for about this much". Charging becomes a background task, like email spam filtering - valuable precisely because you never open it. No dashboard, no login, no notifications for the person in the car; the console exists for the people who operate the grid.

Q10. What data leaves the car? Privacy answer.

Position, state of charge and a pseudonymous rotating token - nothing else. No plates, no names, no trip history pooled across cities: the pseudonymization gateway holds the token-to-session mapping and each region's Center sees only its own vehicles. The SDK is open and auditable, so OEMs and regulators can verify the claim instead of trusting it.

Q11. What if an OEM refuses to ship the SDK?

We do not need every car on day one - the optimization only gets better with each additional agent, and any fleet operator or charging network can deploy the SDK independently of OEMs. The registry proves a vehicle exists before it can claim capacity, so the system is safe even at low adoption. The open SDK is the moat: it is the cheapest path for any manufacturer to offer quantum-optimized charging to its customers.

Risks and pushback

Q12. What breaks first at 100x scale?

Two things, and both have a plan. QUBO size grows with contested slots, not fleet size - the classical pre-filter keeps 2 candidate stations per vehicle, so we attack only the cells that actually collide, and regional Centers shard the problem geographically before federation. Second, QAOA depth: p=1 today, deeper circuits and hardware with more qubits tomorrow - the same QUBO compiles to hardware unchanged.

Q13. Aren't you dependent on quantum hardware that barely exists?

Our layered answer: the production path is classical and always ships; QAOA is measured against a brute-force ground truth every single run, so its value is evidence, not narrative. When hardware improves, we change solver flags, not architecture. Meanwhile the same QUBO runs on annealers and gate machines alike - that is what encoding in the Ising native format buys you.

Q14. Fairness - won't rich neighborhoods charge first?

Fairness in Q Grid is a published number, not a vibe: the waiting penalty weights Rs 260 / 150 / 80 per slot are set by the urgency triage, printed in the repo, and tunable by operators. A low-battery car in any zone outranks a comfortable car in a rich one, and the feasibility guard guarantees no one is scheduled into a full port. Every plan is reproducible from seed 42, so any regulator can re-run our exact scenario.

Q15. Give me your three closing one-liners.

One: "We did not add a single charger - we made every one of them smarter." Two: "Drivers never open an app; charging is a background task." Three: "The cloud was built by people who owned the orchestration layer - Q Grid is that layer for energy, owned by the community that builds it."

Demo flow checklist (90 seconds in the console)

Step	Action	What to say
1	Open dashboard/index.html	"This is the operator console - the community's bridge, not a driver app."
2	KPI row on top	"Minus 21% average wait, zero violations, QAOA within 1.5% of the provable optimum."
3	Gantt of the schedule	"Every car gets one continuous session - nobody gets bounced off a port."
4	Solver cards + curves	"Four solvers, one QUBO, honest baselines - brute force keeps us honest."
5	Fleet table last	"And it is all reproducible: seed 42 - results.json is the audit trail."

Say / don't say

Say this	Not this
"We measure every run against a provable optimum."	"Quantum always wins."
"SA ships the plan in 2 seconds; QAOA upgrades the contested cell."	"We need a quantum computer to run this."
"The community funds and governs it; drivers stay invisible."	"Drivers will love our dashboard."
"At 15 cars the rupees are small - the mechanism is what scales."	"This saves a city money today."

If you only remember three numbers: -21% average wait, 0 violations, QAOA ratio 1.015 against a provable optimum.